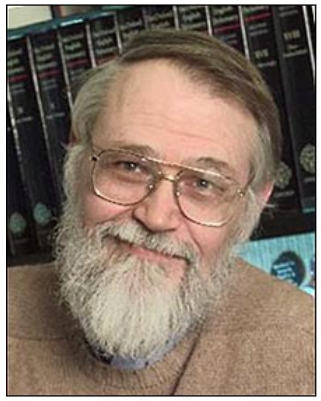
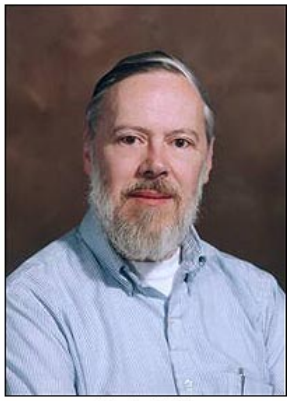
Eine Einführung in die Programmiersprache C

Geschichtliche Entwicklung von C


Brian Kernighan


Dennis Ritchie

- C wurde 1967-1970 von Dennis Ritchie bei Bell Laboratories zur Programmierung des UNIX-Betriebssystems entwickelt
  - Weiterentwicklung der Sprache B (Thompson & Ritchie)
- 1978: Kernighan und Ritchie veröffentlichen das Buch „The Programming Language C“
  - jahrelang der De-facto-Standard (heute K&R-C genannt)
- 1989 : Norm ANSI X3.159-1989 Programming Language C wird verabschiedet (C89, „ANSI C“)
  - 1990 von der ISO als ISO/IEC 9899:1990 übernommen (C90)
- 1999: Einführung des Standards ISO/IEC 9899 (C99, „ISO C“)
  - C wird um einige aus C++ bekannte Standards erweitert
- 2011: weitere Standardisierung durch ISO/IEC 9899:2011 (C11)

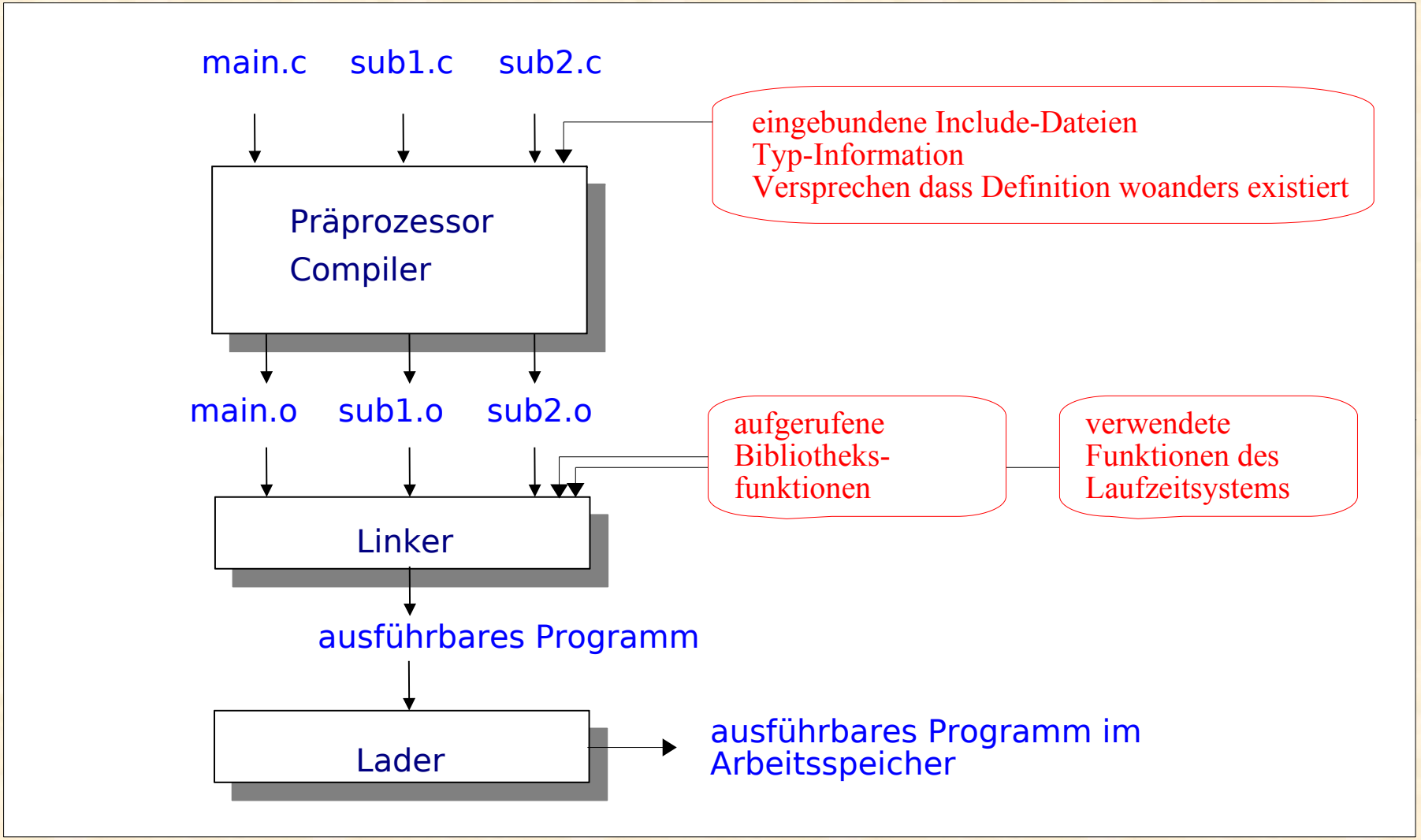
Eigenschaften von C

- höhere Programmiersprache
  - Sequenz, Wiederholung, Auswahl
  - Datentypkonzept: einfach und zusammengesetzt, problemanpassbar
  - Funktionen mit Parameterübergabe
- aber, zugleich auch eine maschinennahe Programmiersprache
  - direkter Zugriff auf Speicherstellen mittels Zeiger und Adress-Arithmetik
  - Register-Variablen, statische Variablen innerhalb von Funktionen
  - direkte Manipulation und Zugriff auf einzelne Bits
- vielfältige Möglichkeiten zum Anfertigen fehlerhafter Programme
  - keine strenge Typkonsistenz (vielfältige automatische Typkonvertierungen)
  - geringe Exaktheit in der Sprachdefinition
  - Zeiger, Adress-Arithmetik, manuelle Speicherverwaltung

Eigenschaften von C

- Portabel
  - Sprachumfang klein (z.B. recht wenig Schlüsselwörter) und übersichtlich, was schnelle und einfache Erstellung von Übersetzern unterstützt
  - auf Computersystemen deshalb auch meist die erste verfügbare Programmiersprache
- Erstellung sehr effizienter Programme
  - C-Quellprogramme lassen sich sehr kompakt aufschreiben
  - C-Compiler erzeugen relativ kompakten und schnell ausführbaren Programmcode
- Multithreaded
  - Möglichkeit zur nebenläufigen Abarbeitung von Programmen
- „Programmierer müssen wissen, was sie tun“

Programmierumgebung: C



Erstellung und Ausführung von Programmen

- Erstellen der Datei hello.c

```
/* Datei: hello.c */
#include <stdio.h>
int main(void){   /* Programm hat (genau) eine main()-Funktion  */
    printf("Hello World");
    return 0;
}
```

- Übersetzen des Quelltextes

```
gcc -std=c99 -Wall -Wextra -pedantic hello.c -o hello
```

- Programmausführung

```
Hello World
```

-std=XXX, legt Sprachstandard fest
-Wall und -Wextra, aktiviert zusätzliche Warnungen
-pedantic, Warnungen für nicht iso-konforme Verwendungen
-pedantic-errors, erzwingt Fehlerausgaben
-o, benennt ausführbares Programm, sonst a.out

Erneuerung durch C99- und C11-Standard

Standard-C99	· Inline-Funktionen, Vereinbarungsanweisungen, Datentypen fester Größe · Typ *long long int* sowie Makros für boolesche und komplexe Zahlen · Arrays variabler Länge, Einzeilige Kommentare // · verbesserte Unterstützung des IEEE 754 für Gleitpunktzahlen · Schlüsselwort *restrict* für Zeiger zur verbesserten Codeoptimierung · Initialisierung einzelner Strukturkomponenten, anonyme Variablen · Unterstützung von Makros mit variabler Parameterlänge · benutzerdefinierte Namen in universelleren Zeichensätzen (wie Unicode)
Standard-C11	· präzisere Kennzeichnung von Unicode (UTF-8, UTF-16, UTF-32) · Anonyme Strukturen und Unions, Funktionen ohne Wiederkehr · Exklusiv-Flag für lesbare Dateien, Ausrichtung von Strukturen im Speicher · statische Auswertung von Prüf-Ausdrücken, generische Typ-Makros · optionales Multithreading und atomare Operationen · optionale Verwendung von IEEE 754 für Gleitpunktzahlen, komplexe Zahlen und Arrays mit variabler Länge (fester Bestandteil in C99) · neue Makros zum Abfragen, ob System gewünschte Funktionalität unterstützt

Standards liefern keine exakte Sprachbeschreibung

- Problem: Verhalten bestimmter korrekter C-Sprachkonstruktionen ist durch Standards nicht festgelegt
- implementierungsabhängiges Verhalten (für korrekte C-Konstruktion)
  - Ergebnis kann sich von Compiler zu Compiler unterscheiden
  - Compiler-Hersteller muss Ergebnis festlegen und dokumentieren
- unspezifiziertes Verhalten (für korrekte C-Konstruktion)
  - Compiler-Hersteller muss sich für bestimmtes Verhalten entscheiden
  - Verhalten muss aber nicht dokumentiert werden
  - Beispiel: Auswertung aktueller Parameter ist nicht festgelegt

    was liefert: a=10; g(a*=5, a+=200);
- undefiniertes Verhalten (für korrekte C-Konstruktion):

  Beispiel: Verhalten bei Nulldivision ist im C-Standard undefiniert